

Studiu Comparativ al Algoritmilor de Reinforcement Learning în Medii Atari

Dragomir Daria Nicoleta (352)

Mihăilă Denisa (352)

Nițe Alexandru (344)

Asavei Roxana (351)

Voica Alex-Ștefan (351)

Rezumat

Proiectul constă în implementarea, evaluarea și analiza comparativă a trei algoritmi fundamentali de Reinforcement Learning: **PPO** (Proximal Policy Optimization), **DQN** (Deep Q-Network) și **C51** (Categorical DQN). Agenții au fost antrenați și testați pe 5 jocuri Atari (Pong, Breakout, Space Invaders, Beam Rider, Freeway.) folosind librăriile **Gymnasium**, **Stable-Baselines3** și implementări proprii în PyTorch.

Cuprins

1	Descrierea Mediului de Lucru (Atari)	2
1.1	Preprocesarea și Spațiul de Stări	2
1.2	Jocurile Utilizate	2
2	Algoritmi Implementați și Fundamente Matematice	2
2.1	DQN (Deep Q-Network)	2
2.1.1	Ecuția Bellman și Loss	3
2.1.2	Componente Cheie în Cod	3
2.2	C51 (Categorical DQN)	3
2.2.1	Conceptul Distribuțional	3
2.2.2	Actualizarea Proiectată (Projection Step)	3
2.3	PPO (Proximal Policy Optimization)	4
2.3.1	Funcția Obiectiv Clipped	4
2.3.2	Componente Cheie în Cod	4
3	Hiperparametri și Configurație Experimentală	4
3.1	Analiza Codului și a Alegerilor de Design	5
4	Rezultate Experimentale	5
4.1	C51	5
4.2	PPO	7
4.3	DQN	8
5	Concluzii	9

1 Descrierea Mediului de Lucru (Atari)

Experimentele au fost realizate folosind mediul ALE (Arcade Learning Environment) prin interfața Gymnasium. Jocurile selectate prezintă provocări diverse din punct de vedere al dinamicii și al recompenselor.

1.1 Preprocesarea și Spațiul de Stări

Pentru toți algoritmii, am aplicat tehnici standard de preprocesare pentru a reduce complexitatea computațională (AtariPreprocessing și VecFrameStack):

- **Grayscale & Resizing:** Imaginile sunt convertite în tonuri de gri și redimensionate la 84×84 pixeli.
- **Frame Stacking:** Starea curentă S_t este compusă din ultimele 4 cadre consecutive. Acest lucru este important pentru a permite agentului să deducă direcția și viteza obiectelor (Markov Property), lucru imposibil dintr-o singură imagine statică.
- **Action Space:** Discret (ex: Stânga, Dreapta, Fire, NoOp).

1.2 Jocurile Utilizate

1. **Pong:** Un joc simetric cu recompensă densă. Este cel mai simplu mediu, ideal pentru verificarea corectitudinii algoritmilor.
2. **Breakout:** Necesită coordonare și planificare pe termen lung (săpatul tunelurilor pentru a maximiza scorul). Recompensa este legată de spargerea cărămizilor.
3. **Space Invaders:** Un mediu mai dinamic cu inamici care se mișcă și trag, necesitând o politică de evitare și atac simultan.
4. **Beam Rider:** Testează capacitatea de reacție rapidă și planificarea traiectoriei pentru a ataca navetele inamicilor.
5. **Freeway:** Testează capacitatea de planificare a traiectoriei pentru a face jucătorul să traverseze străzile fără a fi lovit de mașini.

2 Algoritmi Implementați și Fundamente Matematice

2.1 DQN (Deep Q-Network)

DQN este un algoritm *value-based*, *off-policy*, care extinde Q-Learning prin utilizarea rețelelor neuronale pentru a aproxima funcția de valoare $Q(s, a)$.

2.1.1 Ecuația Bellman și Loss

Obiectivul este minimizarea erorii Bellman (TD Error). În codul nostru, folosim `SmoothL1Loss` (Huber Loss) pentru stabilitate:

$$L(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right] \quad (1)$$

Unde θ sunt parametrii rețelei curente (`policy_net`), iar θ^- sunt parametrii rețelei țintă (`target_net`).

2.1.2 Componente Cheie în Cod

- **Replay Buffer:** Stochează tranzițiile (s, a, r, s', d) pentru a rupe corelația temporală dintre datele de antrenare. Codul utilizează un buffer de dimensiune `BUFFER_SIZE` = 200,000.
- **Target Network:** O copie a rețelei care se actualizează lent. În implementarea noastră folosim *Polyak Averaging* (soft update) definit de hiperparametrul `TARGET_TAU` = 0.005.
- **Mixed Precision (AMP):** Codul utilizează `torch.cuda.amp.GradScaler` pentru a accelera antrenarea pe GPU, reducând consumul de memorie fără a pierde precizie semnificativă.

2.2 C51 (Categorical DQN)

C51 este un algoritm de *Distributional Reinforcement Learning*. Spre deosebire de DQN, care estimează o medie scalară a recompenselor viitoare, C51 estimează întreaga distribuție de probabilitate a acestora.

2.2.1 Conceptul Distribuțional

În loc de $Q(s, a) \in \mathbb{R}$, C51 învață o variabilă aleatoare $Z(s, a)$. Distribuția este modelată printr-un set discret de suporturi (atomi). În codul nostru, variabila `atoms` este definită de parametrii:

- V_{min}, V_{max} : Limitele suportului (ex: -10 la 10).
- $N_{atoms} = 51$: Numărul de puncte discrete.

2.2.2 Actualizarea Proiectată (Projection Step)

Cea mai complexă parte a implementării implică proiecția distribuției Bellman target înapoi pe suportul discret original al rețelei.

$$\Phi \hat{\mathcal{T}} Z_{\theta}(s, a) \quad (2)$$

Codul calculează proiecția distribuției deplasate de recompensă $(r + \gamma z_j)$ și o redistribuie pe atomii vecini folosind interpolare liniară. Aceasta permite rețelei să învețe nu doar valoarea așteptată, ci și varianța/riscul (de ex: în Breakout, diferența dintre a pierde o viață vs a câștiga puncte).

2.3 PPO (Proximal Policy Optimization)

PPO este un algoritm *policy-based* (Actor-Critic), *on-policy*. Este considerat standardul actual datorită stabilității și ușurinței de utilizare. Implementarea a fost realizată folosind `Stable-Baselines3`.

2.3.1 Funcția Obiectiv Clipped

PPO previne actualizările prea mari ale politicii care ar putea destabiliza antrenamentul. Obiectivul maximizat este:

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t) \right] \quad (3)$$

Unde $r_t(\theta)$ este raportul de probabilitate între noua și vechea politică, iar ϵ este hiperparametrul `clip_range` (l-am setat la 0.1).

2.3.2 Componente Cheie în Cod

- **Environments Paralele:** Utilizăm `make_atari_env(n_envs=8)` pentru a colecta experiență de la 8 instanțe simultan, accelerând antrenarea și reducând varianța.
- **Coeficientul de Entropie (`ent_coef=0.01`):** Încurajează explorarea prevenind convergența prematură către o politică deterministă suboptimală.
- **Actor-Critic:** Rețeaua partajează straturile convoluționale, având două ”capete”: unul pentru acțiune (Actor) și unul pentru valoarea stării (Critic).

3 Hiperparametri și Configurație Experimentală

Tabelul de mai jos sumarizează hiperparametrii critici utilizați în codul sursă pentru cele trei metode.

Hiperparametru	Rol Esențial
Learning Rate	Viteza de actualizare a rețelei (balans între convergență și stabilitate).
Batch Size	Numărul de exemple per pas; influențează stabilitatea gradientului.
Discount Factor (γ)	Orizontul de planificare: importanța recompenselor viitoare vs. imediate.
Buffer Size	(Off-Policy) Memoria de experiențe; reduce corelația temporală a datelor.
Update Frequency	Definește intervalul de pași la care se antrenează rețeaua.
Target Update	Sincronizarea rețelei țintă; vitală pentru stabilitatea antrenării.
Exploration	Echilibrul dintre încercarea acțiunilor noi și folosirea celor cunoscute.

Tabela 1: Rolul hiperparametrilor

3.1 Analiza Codului și a Alegerilor de Design

1. **DQN vs C51 (Arhitectură):** În implementarea noastră, C51 modifică ultimul strat *Linear* al rețelei. În timp ce DQN are output `n_actions`, C51 are output `n_actions × n_atoms`. Aceasta crește complexitatea modelului, dar oferă un semnal de antrenare mai bogat.
2. **Stabilitate (PPO):** PPO utilizează `VecNormalize` și `Clip Range`. În cod, am utilizat `ValueLogger` callback pentru a monitoriza estimarea valorii $V(s)$. Dacă $V(s)$ diverge, antrenamentul eșuează. PPO este, în general, mai robust la alegerea hiperparametrilor decât DQN.
3. **Eficiență (DQN/C51):** Implementarea DQN utilizează `frame_skip=4`, ceea ce înseamnă că agentul ia o decizie la fiecare a 4-a ramă. Acest lucru este standard în Atari pentru a accelera antrenarea fără a pierde informație critică.

4 Rezultate Experimentale

4.1 C51

Impactul hiperparametrilor asupra performanței pentru C51 (5 milioane pași):

Config	Breakout (Reactiv)	Beam Rider (Shooter/Nav)	Pong (Fizică/Plan)	Freeway (Evitare Simplă)	Space Invaders (Dens/Dinamic)
Baseline (51, 0.99, 64)	Ok: Reward $\approx$ 50. Convergență medie.	Bun: Reward $\approx$ 1600. Învățare constantă.	Perfect: Reward $\approx$ +21. Converge rapid.	Rezolvat: Reward 21+. Converge instant.	Bun: Reward $\approx$ 400. Stabil.
Atoms = 21 (Redus)	Neutru: Nicio diferență față de 51.	Neutru: Performanță identică.	Neutru: Tot +21.	Neutru: Tot 21+.	Neutru: Reward $\approx$ 400. <i>Concluzie: 21 atomi ajung.</i>
Gamma = 0.90 (Orizont scurt)	Best: Reward > 60. Favorizează reacția.	Acceptabil: Ușoară scădere ($\approx$ 1500).	Worst: Se blochează la +15.	Bun: Fiind pur reactiv, funcționează perfect.	Slab: Scade scorul. Agentul nu curăță valurile eficient.
Batch = 128 (Mare)	Lent: FPS scade, reward stabil.	Stabil: Fără câștig de performanță.	Stabil: Converge, dar mai lent (timp).	Stabil: Reward max, FPS mai mic.	Instabil: Reward oscilant ($\approx$ 300), FPS mic.
LR = 1e-4 (Mic)	Stabil: Q-values curate, fără zgomot.	Robust: Convergență gură $\approx$ 1700.	Sigur: Atinge +21 și mai lent, dar stabil.	Excelent: Curbă lină, stabilitate max.	Bun: Creștere lentă dar sigură spre 400.

Tabela 2: Sinteza influenței hiperparametrilor C51 pe 5 medii Atari

Concluzii Generale ale Analizei Comparative:

Pe baza rezultatelor din Tabelul 2, se pot desprinde următoarele observații fundamentale despre comportamentul algoritmului C51:

- **Complexitatea Distribuției (N_{atoms}):** Reducerea numărului de atomi de la 51 la 21 a avut un impact neglijabil asupra performanței în toate cele 5 medii testate. Acest lucru sugerează că beneficiul principal al C51 vine din *natura distribuțională* a algoritmului (capacitatea de a modela incertitudinea), și nu neapărat din granularitatea fină a distribuției. O configurație cu 21 de atomi este mai eficientă computațional fără a sacrifica scorul.
- **Sensibilitatea la Orizontul de Planificare (γ):** Factorul de discount a demonstrat cea mai mare varianță în impact, dictată strict de dinamica jocului:
 - În jocurile pur *reactive* (Breakout, Freeway), unde acțiunea corectă depinde imediat de poziția curentă a mingii/obstacolului, un $\gamma = 0.90$ a fost benefic sau neutru.
 - În jocurile care necesită *planificare pe termen lung* sau predicția traiectoriilor fizice complexe (Pong), reducerea γ a fost catastrofală, împiedicând agentul să învețe returnul mingii.
- **Stabilitatea Antrenării (Batch Size & Learning Rate):** Creșterea dimensiunii batch-ului la 128 nu a adus beneficii tangibile, ci dimpotrivă, a încetinit antrenarea (FPS scăzut) și a indus instabilitate în medii complexe precum Space Invaders. În schimb, reducerea ratei de învățare ($1e^{-4}$) a garantat o convergență mai lentă, dar extrem de stabilă, eliminând colapsurile bruște ale politicii (policy collapse).
- **Gradul de Dificultate:** Freeway a fost cel mai ușor mediu ("rezolvat" rapid), în timp ce Space Invaders și Beam Rider au necesitat un echilibru mai fin al hiperparametrilor pentru a maximiza scorul, fiind sensibile la zgomotul din gradient.

4.2 PPO

Config	Beam Rider (Shooter)	Breakout (Precizie)	Freeway (Navigare)	Pong (Reactiv)	Space Invaders (Dinamic)
Baseline ($2.5e^{-4}$, 256)	Slab (Fig 1): Reward ≈ 4 . Mult sub DQN. 1M pași sunt insuficienți pentru PPO aici.	Best: Reward ≈ 20 . Creștere liniară și stabilă.	Rezolvat: Reward 21+. Convergență rapidă (400k pași).	Perfect: Reward +21. Converge curat la 700k pași.	Mediocr: Reward ≈ 9 . Creștere lentă și zgomotoasă.
Low LR ($5e^{-5}$)	Stagnare (Fig 2): Reward ≈ 3.5 . Învățare prea lentă, abia depășește random.	Fail: Reward < 5 . Underfitting sever.	Robust: Reward 21+. Învăță mai lent, dar sigur.	Lent: Reward ≈ -19 . Abia începe să învețe după 1M pași.	Stagnare: Reward ≈ 5 . Prea lent pentru inamici dinamici.
High LR ($1e^{-3}$)	Eșec (Fig 3): Reward ≈ 3 . Politica nu progresaază deloc.	Instabil: Reward $(N/A - \text{Instabil})$ plafonat la 10. Varianță mare.		Surprinzător: Reward +20. Învăță rapid reacția brută.	Zgomotos: Reward ≈ 9 . Loss-ul explodează.
Small Batch (64)	Instabil (Fig 4): Reward ≈ 4 . Loss-ul criticului este foarte zgomotos.	Slab: Reward ≈ 9 . Gradient haotic.	Excelent: Reward 25+. Zgomotul ajută explorarea.	Ok: Reward ≈ 18 . Converge, dar cu oscilații.	Instabil: Reward ≈ 8 . Critic loss neregulat.
Large Batch (512)	Stabil (Fig 5): Reward ≈ 6 . Cea mai bună variantă, dar tot slabă comparativ cu DQN.	Bun: Reward ≈ 18 . Foarte stabil.	Lent: Convergență mai lentă în timp real.	Lent: Reward ≈ -18 . Crește mult mai greu.	Slab: Reward ≈ 6 . Updates prea rare pentru reacție.

Tabela 3: Sinteza Performanței PPO pe 5 Medii Atari

Analiza Detaliată a Rezultatelor și Observații Cheie

Pe baza graficelor generate și a datelor colectate, se disting următoarele tendințe majore în comportamentul algoritmului PPO:

- Breakout vs. Pong (Impactul Ratei de Învățare):**
 În mediul *Breakout*, utilizarea unui Learning Rate ridicat (10^{-3}) a degradat sever performanța, plafonând recompensa medie la aproximativ 10. Acest lucru se explică prin faptul că jocul necesită o precizie ridicată pentru săparea "tunelurilor" fine. În contrast, pe mediul *Pong*, același LR ridicat a funcționat surprinzător de bine, agentul atingând scorul maxim (+21) chiar mai rapid decât în configurația baseline. *Concluzie:* Jocurile pur reactive (*Pong*) tolerează actualizări agresive ale politicii, în timp ce jocurile care necesită precizie spațială și secvențialitate fină (*Breakout*) necesită pași de actualizare mai mici.
- Space Invaders – Limitări structurale ale PPO:**
 Indiferent de configurația de hiperparametri testată (Batch Size mic/mare, LR divers), PPO a întâmpinat dificultăți majore în a depăși un prag al recompensei de 10 – 12 puncte în primele milioane de pași. Comparativ cu algoritmul C51 (care tinde să exceleze pe acest mediu datorită perspectivei distribuționale asupra riscului), PPO, fiind *on-policy*, suferă din cauza complexității vizuale (mulți inamici mici) și a sparsității recompenselor, nereușind să dezvolte o strategie eficientă de "curățare" a ecranului.
- Batch size (Freeway vs. Restul):**
Freeway este singurul mediu care a beneficiat semnificativ de reducerea dimensiunii

lotului (Batch Size = 64), obținând scoruri record de 25+. Ipoteza este că zgomotul statistic indus de eșantionarea pe loturi mici a acționat ca un mecanism de explorare benefic, prevenind blocarea agentului într-o stare de așteptare pasivă a ”momentului perfect” pentru traversare. În schimb, pe mediile *Pong* și *Space Invaders*, reducerea batch-ului a introdus doar instabilitate și divergență în funcția de pierdere a criticului (*Critic Loss*).

4.3 DQN

Config	Beam Rider (Shooter/Nav)	Space Invaders (Dinamic/Haotic)	Breakout (Planificare/Tunnel)	Freeway (Traversare)	Pong (Reactiv/ZeroSum)
Baseline ($5e^{-5}$, 64)	Optim: Reward $\approx$ 1100. Creștere liniară stabilă a Q-values.	Mediu: Reward $\approx$ 300. Convergență zgomotoasă (“noisy”).	Lent: Reward $\approx$ 30. Nu reușește strategia de tunel.	Lent: Reward $<$ 15. Prea precaut, Q-values $\approx$ 0.	Slab: Reward $\approx$ -19. Învățare greoaie, corelație slabă.
High LR ($2.5e^{-4}$)	Eșec: Spike Q-Val (> 75) apoi colaps. Catastrophic forgetting.	Instabil: Reward $<$ 200. Oscilații mari în Loss.	Critic: Loss explodează. Agentul nu controlează paleta.	Best: Reward 21+. Soluție instantă (1k ep).	Eșec Total: Blocați la -21. Gradienții destabilizează politica.
Large Batch (128)	Stabil: Reward bun. Varianță redusă, dar mai lent.	Robust: Ajută la stabilitatea traiectoriilor inamicilor.	Neutru: Nicio îmbunătățire strategică (fără tunel).	Stabil: Converge sigur, dar mai lent decât High LR.	Lent: Reward $\approx$ -15. Varianța redusă nu rezolvă sparsity.
Buffer 300k (Memorie)	Bun: Previne overfitting-ul pe pattern-uri locale.	Benefic: Reward consistent. Decorează stările similare.	Esențial: Păstrează tranzițiile rare. Q-val explodează pozitiv.	Interessant: Dip în Q-values apoi recuperare.	Promițător: Reward spre -10. Cea mai bună pantă de învățare.
Concluzie	DQN domină mediile structurate.	Dificil din cauza stocasticității.	Necesită Buffer mare pentru strategie.	Excepție: LR mare (problemă simplă).	Preferă Inferior C51/PPO (lent).

Tabela 4: Performanța DQN

Concluzii și Analiză DQN

Pe baza rezultatelor experimentale centralizate în Tabelul 4, se pot desprinde următoarele concluzii esențiale privind comportamentul algoritmului DQN în comparație cu cerințele mediilor Atari:

- **Sensibilitate Critică la Hiperparametri (Learning Rate):** Spre deosebire de algoritmi moderni (ex: PPO care utilizează *clipping*), DQN este predispus la instabilitate majoră dacă pasul de învățare este prea mare. Experimentele cu $LR = 2.5 \times 10^{-4}$ au demonstrat fenomenul de *catastrophic forgetting* (în special în Breakout și Beam Rider), unde estimările Q (Q-Values) cresc nerealist, ducând la degradarea rapidă a politicii agentului.
- **Impactul Replay Buffer-ului asupra Strategiei:** Dimensiunea memoriei este direct corelată cu capacitatea de planificare pe termen lung. În *Breakout*, creșterea buffer-ului la 300.000 de tranziții a fost singura modificare care a permis agentului să ”rețină” secvențele rare care duc la spargerea zidului superior (strategia de tunel), lucru imposibil cu un buffer mic care suprascrisese rapid experiențele vechi.
- **Limitări în Medii Stocastice:** DQN performează optim în medii cu fizică predicibilă și recompense dense (Beam Rider, Freeway). Totuși, în medii aglomerate sau

haotice (Space Invaders), reducerea întregii distribuții a recompenselor viitoare la o singură valoare scalară (medie) limitează performanța, agentul având dificultăți în a distinge între acțiuni sigure și acțiuni riscante, aspect adresat mai eficient de C51.

5 Concluzii

În urma rulării experimentelor pentru 5 milioane de pași, s-au observat următoarele tendințe:

- **DQN** tinde să aibă o convergență mai lentă și o varianță mare a recompenselor (graficele arată oscilații), specific algoritmilor off-policy cu buffer limitat.
- **C51** arată o îmbunătățire a stabilității și a scorului final față de DQN standard, confirmând ipoteza că modelarea distribuției ajută în medii stocastice sau complexe.
- **PPO** este cel mai stabil și ușor de paralelizat (datorită celor 8 medii simultane), atingând un scor rezonabil într-un timp mai scurt.